

Unit 2: Regular Languages, Properties & Minimization

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Regular Expressions: Inductive definition, operators, precedence, identities and algebraic laws. Equivalence of RE & Finite Automata: Converting RE to ϵ -NFA using Thompson's construction. Converting FA to Regular Expressions: Sta...

Table of Contents

Regular Expressions (RE) & Algebraic Laws	p. 2
Equivalence: RE to Finite Automata (Thompson's Construction)	p. 3
Converting Finite Automata to Regular Expressions	p. 5
The Pumping Lemma for Regular Languages	p. 9
Closure Properties of Regular Languages	p. 12
Decision Properties of Regular Languages	p. 14
Minimization of DFA (The Table-Filling Algorithm)	p. 15
One-Page Rapid Revision Sheet	p. 17
Important Examination Questions	p. 18
Solved Questions from Past CIE-1 & SEE Examination Papers	p. 21
Practice Problems with Step-by-Step Solutions	p. 23

Regular Expressions (RE) & Algebraic Laws

§ 1

A **regular expression** provides an algebraic, declarative specification of a language accepted by a finite automaton. While finite automata describe machines that recognize strings step by step, regular expressions generate the same set of strings algebraically.

INDUCTIVE DEFINITION OF REGULAR EXPRESSIONS:

Let Σ be a given alphabet. The regular expressions over Σ and the languages they denote are defined inductively as follows:

Basis Clauses:

1. $\emptyset$ is a regular expression denoting the empty language: $L(\emptyset) = \emptyset$.
2. ϵ is a regular expression denoting the language containing only the empty string:
 $L(\epsilon) = \{\epsilon\}$.
3. For each symbol $a \in \Sigma$, $\mathbf{a}$ is a regular expression denoting the singleton language:
 $L(\mathbf{a}) = \{a\}$.

Inductive Clauses: If R and S are regular expressions denoting languages $L(R)$ and $L(S)$ respectively, then:

1. **Union (Alternate):** $(R + S)$ or $(R \mid S)$ is a regular expression denoting
 $L(R + S) = L(R) \cup L(S)$.
2. **Concatenation:** $(R \cdot S)$ or (RS) is a regular expression denoting $L(RS) = L(R) \cdot L(S) = \{uv \mid u \in L(R), v \in L(S)\}$.
3. **Kleene Star (Closure):** (R^*) is a regular expression denoting $L(R^*) = (L(R))^* = \bigcup_{i=0}^{\infty} (L(R))^i$, where $(L(R))^0 = \{\epsilon\}$.

Operator Precedence

In the absence of explicit parentheses, operators bind according to strict precedence:

1

Highest: Kleene Star (*) – binds tightest to its immediate preceding operand.

2

Middle: Concatenation (.) – evaluated before union.

3

Lowest: Union (+ or |) – evaluated last.

Example: $01^* + 10$ is strictly parenthesized as $((0(1^*)) + (10))$.

Important Algebraic Identities

Identity Name	Algebraic Law	Semantic Explanation
Commutativity of +	$R + S = S + R$	Set union is symmetric.
Associativity of +	$(R + S) + T = R + (S + T)$	Grouping does not alter set union.
Associativity of ·	$(RS)T = R(ST)$	String concatenation is strictly associative.
Distributivity	$R(S + T) = RS + RT$ $(S + T)R = SR + TR$	Concatenation distributes from left and right over union.
Identity Elements	$R + \emptyset = \emptyset + R = R$ $R\epsilon = \epsilon R = R$	$\emptyset$ is additive identity; ϵ is multiplicative identity.
Annihilator	$R\emptyset = \emptyset R = \emptyset$	Concatenating any language with the empty set yields no strings.
Idempotence of +	$R + R = R$	Adding a set to itself does not introduce new strings.
Closure Properties	$(R^*)^* = R^*$ $\emptyset^* = \epsilon$ $\epsilon^* = \epsilon$ $R^+ = RR^* = R^*R$	Repeated closure does not expand the language beyond one star.
Shifting Identity	$(R + S)^* = (R^*S^*)^* = (R^* + S^*)^*$	Any interleaving of R and S strings is captured equally.

COMMON MISTAKE ALERT:

Concatenation is NOT commutative: $RS \neq SR$ in general! For example, if $R = a$ and $S = b$, then $RS = \{ab\}$, whereas $SR = \{ba\}$. Do not swap the order of concatenation in algebraic simplifications.

QUICK RECALL

- Basis: $\emptyset, \epsilon, a$. Inductive operators: $+, \cdot, *$.
- Precedence: $* > \cdot > +$.
- $R\emptyset = \emptyset$, but $R\epsilon = R$. $\emptyset^* = \epsilon$. Concatenation does NOT commute.

Equivalence: RE to Finite Automata (Thompson's Construction)

§ 2

Kleene's Theorem establishes that a language is regular if and only if it is denoted by a regular expression and recognized by a finite automaton. Thompson's Construction converts any regular expression R into an equivalent ϵ -NFA with exactly **one start state** and **one final state**.

THOMPSON'S STRUCTURAL INDUCTION RULES:

For any regular expression R , the constructed ϵ -NFA $M(R)$ satisfies:

- $M(R)$ has exactly one start state with no incoming transitions.
- $M(R)$ has exactly one final state with no outgoing transitions.
- Number of states is at most $2 \times$
(number of symbols and operators in R).

Structural Construction Gadgets

Below are the canonical Thompson gadgets for base elements and operators:

Thompson's Construction Structural Gadgets

1. Symbol 'a'



2. Concatenation: R

1. R_2 (Merge f_1 into s_2 or ϵ) $N(R_1) \in N(R_2)$ 3. Union: $R_1 + R_2$ $s_0 \in N(R_1) \in N(R_2)$ f_0 4. Kleene

Closure: R^* $s_0 \in N(R)$ ϵ $f_0 \in$ (loop back) ϵ (bypass / empty string)

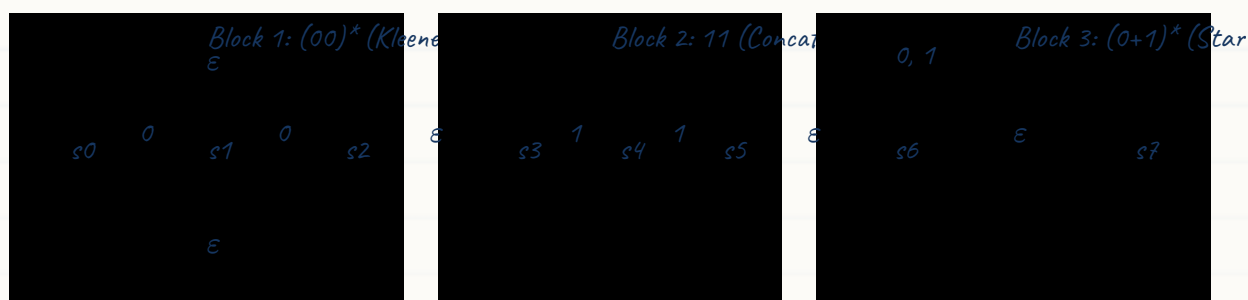


Figure 2.2: Thompson's ϵ -NFA Construction for $(00)^* 11 (0+1)^*$

Ramaiah CIE-1 Q2(b) Solved: Demonstrates inductive composition of Kleene Star $((00)^*)$, Concatenation (11) , and Star of Union $((0+1)^*)$.

QUICK RECALL

- Thompson's construction yields an ϵ -NFA with exactly 1 start and 1 final state.
- Star adds 2 new states and 4 ϵ -transitions (forward into body, out of body, loopback, and zero-iteration bypass).
- Union adds 2 new states and 4 ϵ -transitions (splitting into branches and joining into final).

Converting Finite Automata to Regular Expressions

§ 3

Two established methods extract a regular expression from any finite automaton: the **State Elimination Method** and **Arden's Theorem**. Both are heavily examined in university evaluations.

Method A: State Elimination Method

This method converts a DFA or NFA into a Generalized Transition Graph (GTG) where edge labels are regular expressions, and then eliminates states one by one until only the start and final states remain.

STATE ELIMINATION ALGORITHM:**1. Standardize Start and Final States:**

- If the initial state has incoming transitions, create a new start state s_{new} with an ϵ -transition to the old start state.
- If there are multiple final states or the final state has outgoing transitions, create a new final state f_{new} and connect all old final states to f_{new} via ϵ -transitions.
Remove final status from old final states.

2. Collapse parallel edges: For any pair of states p and q , replace multiple transitions $R_1, R_2, \dots$ with a single transition labeled $(R_1 + R_2 + \dots)$.**3. Eliminate Intermediate State k :** For every pair of states p (predecessor of k) and q (successor of k), update the direct edge label from p to q :

$$\alpha_{pq}^{new} = \alpha_{pq}^{old} + \alpha_{pk} \cdot (\alpha_{kk})^* \cdot \alpha_{kq}$$

where α_{kk} is the self-loop regular expression on state k (or $\emptyset$ if no self-loop, yielding $\emptyset^* = \epsilon$).

4. Repeat until only s_{new} and f_{new} remain. The final label on the edge (s_{new}, f_{new}) is the required regular expression.Method B: Arden's TheoremARDEN'S THEOREM STATEMENT:

Let P and Q be two regular expressions over Σ . If P does not contain the null string ϵ (P has no ϵ), then the linear equation in R :

$$R = Q + RP$$

has a **unique solution** given by:

$$R = QP^*$$

FORMAL PROOF OF ARDEN'S THEOREM:**Part 1: Proof of Existence (Substitution)**

Substitute $R = QP^*$ into the right-hand side:

$$\text{RHS} = Q + (QP^*)P = Q(\epsilon + P^*P)$$

Since $\epsilon + P^*P = P^*$, we have $\text{RHS} = QP^* = \text{LHS}$. Thus $R = QP^*$ satisfies the equation.

Part 2: Proof of Uniqueness (Iterative Expansion)

Repeatedly substitute $R = Q + RP$ into itself:

$$R = Q + (Q + RP)P = Q + QP + RP^2$$

$$R = Q + QP + QP^2 + \dots + QP^k + RP^{k+1}$$

$$R = Q(\epsilon + P + P^2 + \dots + P^k) + RP^{k+1}$$

As $k \rightarrow \infty$, since $\epsilon \notin P$, any string in RP^{k+1} has length at least $k + 1$. For any finite length string w , by choosing $k > |w|$, w cannot belong to RP^{k+1} . Therefore, $R = Q \cdot P^*$, proving the solution is unique.

WORKED NUMERICAL · EXAM LEVEL [10 MARKS · SEE/CIE]

Extract Regular Expression Using Arden's Theorem

Consider the DFA with states q_1, q_2, q_3 , start state q_1 , and accept state q_3 , defined by the transitions:

- $\delta(q_1, 0) = q_2, \delta(q_1, 1) = q_1$
- $\delta(q_2, 0) = q_1, \delta(q_2, 1) = q_3$
- $\delta(q_3, 0) = q_3, \delta(q_3, 1) = q_2$

Step 1: Formulate the characteristic state equations

Express each state as the union of incoming transitions, adding ϵ to the start state equation:

$$q_1 = q_1 1 + q_2 0 + \epsilon \quad \text{— (Eq. 1)}$$

$$q_2 = q_1 0 + q_3 1 \quad \text{— (Eq. 2)}$$

$$q_3 = q_2 1 + q_3 0 \quad \text{— (Eq. 3)}$$

Step 2: Solve Equation 3 for q_3 in terms of q_2 using Arden's Theorem

Equation 3 is: $q_3 = q_2 1 + q_3 0$. This is of the form $R = Q + RP$ where $R = q_3, Q = q_2 1, P = 0$. Since $\epsilon \notin 0$, by Arden's Theorem:

$$q_3 = (q_2 1) 0^* = q_2 1 0^* \quad \text{— (Eq. 4)}$$

Step 3: Substitute Eq. 4 into Eq. 2 to eliminate q_3

$$q_2 = q_1 0 + (q_2 1 0^*) 1 = q_1 0 + q_2 (1 0^* 1)$$

Apply Arden's Theorem ($R = q_2, Q = q_1 0, P = 1 0^* 1$):

$$q_2 = q_1 0 (1 0^* 1)^* \quad \text{— (Eq. 5)}$$

Step 4: Substitute Eq. 5 into Eq. 1 to solve for q_1

$$q_1 = q_1 1 + [q_1 0 (1 0^* 1)^*] 0 + \epsilon = q_1 (1 + 0 (1 0^* 1)^* 0) + \epsilon$$

Apply Arden's Theorem ($R = q_1, Q = \epsilon, P = 1 + 0 (1 0^* 1)^* 0$):

$$q_1 = \epsilon (1 + 0 (1 0^* 1)^* 0)^* = (1 + 0 (1 0^* 1)^* 0)^*$$

Step 5: Compute regular expression for the accepting state q_3

Using Eq. 5 and Eq. 4: $q_3 = q_2 1 0^* = [q_1 0 (1 0^* 1)^*] 1 0^*$. Substituting q_1 :

$$\mathbf{R} = (1 + 0 (1 0^* 1)^* 0)^* \cdot 0 (1 0^* 1)^* 1 0^*$$

Interpretation: Any number of loops from q_1 to itself, followed by 0 to reach q_2 , any number of round-trips to q_3 and back ($1 0^* 1$), and a final transition $1 0^*$ into the accept state q_3 .

QUICK RECALL

- State Elimination: $\alpha_{pq}^{new} = \alpha_{pq} + \alpha_{pk}(\alpha_{kk})^* \alpha_{kq}$. Always isolate start and final states first.
- Arden's theorem: $R = Q + RP \implies R = QP^*$ (valid when $\epsilon \notin P$).
- Always add $+\epsilon$ to the equation of the START state.

The Pumping Lemma for Regular Languages

§ 4

The **Pumping Lemma** is a powerful theoretical tool used exclusively to prove that a given language is **NOT regular**. It is derived from the Pigeonhole Principle: any finite automaton recognizing an infinite language must eventually repeat a state when processing a sufficiently long string.

FORMAL THEOREM: PUMPING LEMMA:

Let L be a regular language. Then there exists a constant integer $p \geq 1$ (called the **pumping length**), depending only on L , such that for every string $w \in L$ with $|w| \geq p$, w can be broken into three substrings $w = xyz$ such that:

1. **Length bound on head:** $|xy| \leq p$
2. **Non-empty pump:** $|y| \geq 1$ (i.e., $y \neq \epsilon$)
3. **Pumping condition:** For all integers $i \geq 0$, the pumped string $xy^iz \in L$.

Intuition & Proof Idea

If L is regular, there exists a DFA $M = (Q, \Sigma, \delta, q_0, F)$ accepting L with $p = |Q|$ states. When M consumes a string $w = a_1 a_2 \dots a_m$ of length $m \geq p$, the computation passes through a sequence of $m + 1$ states: $r_0, r_1, r_2, \dots, r_m$. Since $m + 1 > p$, by the **Pigeonhole Principle**, at least two states among the first $p + 1$ states ($r_0, \dots, r_p$) must be identical, say $r_j = r_k$ with $0 \leq j < k \leq p$.

This creates a loop labeled by the substring $y = a_{j+1} \dots a_k$. Since the machine is in the exact same state at the beginning and end of y , the loop can be traversed 0 times ($i = 0$), 1 time ($i = 1$), or i times ($i \geq 2$), and the machine will still reach the same accepting state!

Pumping Lemma Automata State Loop Mechanism

9

$0 \leq i \leq j$ $r_j = r_k$ y ($|y| \geq 1, |xy| \leq p$) z $r_m \in F$ Traversing the y -loop i times consumes string $x y^i z$, which must land on the same accepting state r_m !

Adversarial Game Framework

To avoid logical pitfalls, treat the proof by contradiction as a 4-step game against an adversary (the "Demon"):

1

Assumption: Assume L is regular. The adversary chooses the pumping length p (you don't know p , but $p \geq 1$).

2

String Selection: You pick a clever string $w \in L$ parameterized by p such that $|w| \geq p$.

3

Division: The adversary partitions $w = xyz$ subject to $|xy| \leq p$ and $|y| \geq 1$.

4

Contradiction: You pick an integer $i \geq 0$ such that $xy^iz \notin L$, defeating the adversary across all possible partitions of y . Hence L cannot be regular.

CONTRADICTION PROOF 1 · STANDARD EXAM QUESTION [8 MARKS]

Prove that $L = \{a^n b^n \mid n \geq 0\}$ is NOT Regular

Step 1: Assumption: Assume for contradiction that L is regular. Let $p \geq 1$ be the pumping length given by the Pumping Lemma.

Step 2: Choose String w : Let $w = a^p b^p$. Note that $w \in L$ and $|w| = 2p \geq p$.

Step 3: Analyze decomposition $w = xyz$:

- Condition 1 states $|xy| \leq p$. Since the first p characters of w are all a 's, xy must consist exclusively of a 's.
- Condition 2 states $|y| \geq 1$. Therefore, $y = a^k$ for some integer $k \geq 1$.
- Hence, $x = a^j$, $y = a^k$, $z = a^{p-j-k} b^p$, where $j \geq 0$, $k \geq 1$, and $j + k \leq p$.

Step 4: Pump with $i = 2$:

Consider the pumped string $w' = xy^2z$:

$$w' = xy^2z = a^j a^{2k} a^{p-j-k} b^p = a^{p+k} b^p$$

Since $k \geq 1$, $p + k > p$. The number of a 's in w' is $p + k$, while the number of b 's remains p . Since the number of a 's does not equal the number of b 's, $w' \notin L$.

This contradicts Condition 3 of the Pumping Lemma ($xy^i z \in L$ for all $i \geq 0$). Thus, the initial assumption must be false, and $L = \{a^n b^n \mid n \geq 0\}$ is NOT regular.

CONTRADICTION PROOF 2 · NON-LINEAR EXPONENT [8 MARKS]

Prove that $L = \{0^{n^2} \mid n \geq 1\}$ is NOT Regular

Step 1: Assume L is regular with pumping length $p \geq 1$.

Step 2: Choose $w = 0^{p^2} \in L$. Length $|w| = p^2 \geq p$.

Step 3: Partition $w = xyz$ with $|xy| \leq p$ and $|y| = k \geq 1$. Since the alphabet is unary $\{0\}$, $y = 0^k$ where $1 \leq k \leq p$.

Step 4: Pump with $i = 2$: Let $w' = xy^2z$. The length of w' is:

$$|w'| = |xyz| + |y| = p^2 + k$$

Now evaluate the bounds on this new length:

- Since $k \geq 1$, $|w'| = p^2 + k > p^2$.
- Since $k \leq p$, $|w'| = p^2 + k \leq p^2 + p < p^2 + 2p + 1 = (p + 1)^2$.

Therefore, we have strictly:

$$p^2 < |w'| < (p + 1)^2$$

The length of w' lies strictly between two consecutive perfect squares p^2 and $(p + 1)^2$.

Hence, $|w'|$ cannot be a perfect square, meaning $w' \notin L$.

Contradiction! Hence, $L = \{0^{n^2} \mid n \geq 1\}$ is NOT regular.

EXAM STRATEGY: HOW TO PROVE A LANGUAGE IS REGULAR VS NOT REGULAR:

To prove a language **IS regular**: construct a DFA, NFA, or Regular Expression. NEVER use the Pumping Lemma to prove a language is regular!

To prove a language **is NOT regular**: use the Pumping Lemma or closure under intersection with a known regular language.

QUICK RECALL

- Pumping Lemma conditions: $|xy| \leq p$, $|y| \geq 1$, and $\forall i \geq 0, xy^iz \in L$.
- Pumping down ($i = 0$) or pumping up ($i = 2$) creates a length or symbol mismatch.
- Pumping Lemma can ONLY prove non-regularity, never regularity.

Closure Properties of Regular Languages

§ 5

A class of languages is **closed** under an operation if applying that operation to languages within the class always produces another language within the same class. Regular languages enjoy

Operation	Formal Notation	Proof Technique / Construction
Union	$L_1 \cup L_2$	Thompson's construction: branch with ϵ to M_1 and M_2 . Or Product DFA: $F_{prod} = (F_1 \times Q_2) \cup (Q_1 \times F_2)$.
Concatenation	$L_1 \cdot L_2$	Thompson's construction: connect final state of M_1 to start state of M_2 via ϵ .
Kleene Closure	L^*	Thompson's construction: add bypass and loopback ϵ -transitions.
Complement	$\overline{L} = \Sigma^* - L$	Take a <u>total</u> DFA $M = (Q, \Sigma, \delta, q_0, F)$ for L . Complement DFA is $M' = (Q, \Sigma, \delta, q_0, Q - F)$ (invert final and non-final states). <u>Must be a DFA, not NFA!</u>
Intersection	$L_1 \cap L_2$	De Morgan's Law: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$. Or Cross-Product DFA construction where $F_{prod} = F_1 \times F_2$.
Difference	$L_1 - L_2$	$L_1 - L_2 = L_1 \cap \overline{L_2}$. Since regular languages are closed under complement and intersection, they are closed under difference.
Reversal	$L^R = \{w^R \mid w \in L\}$	Reverse all transition arrows in the FA, swap start and final states (introduce a single new start state if multiple finals).
Homomorphism	$h(L) = \{h(w) \mid w \in L\}$	Replace each symbol a on transitions with string $h(a)$. The resulting transition graph is an ϵ -NFA.
Inverse Homomorphism	$h^{-1}(L) = \{w \mid h(w) \in L\}$	Define DFA M' with $\delta'(q, a) = \hat{\delta}(q, h(a))$. Start and final states remain unchanged.

CRITICAL EXAM GOTCHA: COMPLEMENT OF NFA:

You **cannot** complement an NFA simply by swapping its accepting and non-accepting states! Because an NFA can have multiple computation paths for the same string, swapping accept states does not compute the complement language. You **MUST** first convert the NFA to an equivalent DFA (ensuring all dead states are included), and only then swap final and non-final states.

QUICK RECALL

- Regular languages are closed under:
 $\cup, \cap, \cdot, *, \text{complement}, -, \text{reversal}, \text{homomorphism}, h^{-1}$.
- Complement: invert accept states on a **complete DFA** (never an NFA directly).
- Intersection via De Morgan's Law: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$.

Decision Properties of Regular Languages**§ 6**

A property of a language family is **decidable** if there exists an algorithm (a Turing machine that halts on all inputs) that answers yes or no to questions regarding that property. For regular languages, almost all standard properties are fully decidable.

<i>Decision Problem</i>	<i>Question Asked</i>	<i>Deciding Algorithm & Time Complexity</i>
Membership Problem	Given DFA M and string w , is $w \in L(M)$?	Simulate M on w symbol by symbol starting from q_0 . Check if final state is in F . Complexity: $O(w)$ time.
Emptiness Problem	Given DFA M , is $L(M) = \emptyset$?	Perform Graph Reachability (BFS/DFS) starting from q_0 in the state transition graph. If no state in F is reachable, $L(M) = \emptyset$; otherwise non-empty. Complexity: $O(Q + E)$.

Decision Problem	Question Asked	Deciding Algorithm & Time Complexity
Finiteness Problem	Given DFA M , is $L(M)$ finite or infinite?	Method 1: Cycle Detection: Eliminate all unreachable states and states that cannot reach any final state. Then check for directed cycles using DFS. If a cycle exists, $L(M)$ is infinite; else finite. Method 2: Length Bounds: Test if M accepts any string w such that $n \leq w < 2n$, where $n = Q$. If yes, infinite; if none, finite.
Equivalence Problem	Given DFAs M_1, M_2 , is $L(M_1) = L(M_2)$?	Method 1: Symmetric Difference: Construct $L_{sym} = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$. Run Emptiness test on L_{sym}. $L_1 = L_2 \iff L_{sym} = \emptyset$. Method 2: Minimization: Minimize both M_1 and M_2. Check if minimal DFAs are isomorphic up to state renaming.
Subset / Containment	Is $L(M_1) \subseteq L(M_2)$?	Test if $L_1 - L_2 = \emptyset$, which is equivalent to testing whether $L(M_1) \cap \overline{L(M_2)} = \emptyset$.

QUICK RECALL

- Membership: simulate DFA in $O(|w|)$.
- Emptiness: BFS/DFS to check reachability of final states from q_0 .
- Finiteness: check for cycles on paths from q_0 to F , or test strings of length $n \leq |w| < 2n$.
- Equivalence: $(L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2) = \emptyset$, or check minimal DFA isomorphism.

Minimization of DFA (The Table-Filling Algorithm)

§ 7

A Deterministic Finite Automaton may contain redundant states that perform identical functional duties. The **Myhill-Nerode Theorem** guarantees that for every regular language, there exists a **unique minimum-state DFA** (up to state relabeling). The **Table-Filling Algorithm** (also known as Moore's / Hopcroft's state-marking algorithm) finds this minimal DFA.

EQUIVALENT VS DISTINGUISHABLE STATES:

Two states $p, q \in Q$ are said to be **equivalent** (indistinguishable, denoted $p \equiv q$) if for all strings $w \in \Sigma^*$:

$$\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F$$

Conversely, states p and q are **distinguishable** if there exists at least one string $w \in \Sigma^*$ (called a distinguishing witness string) such that exactly one of $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$ is in F .

TABLE-FILLING ALGORITHM (STEP-BY-STEP):

1. **Step 0: Remove Unreachable States:** Perform BFS/DFS from q_0 . Discard any state that cannot be reached from the start state.
2. **Step 1: Construct the Lower-Triangular Table:** Create a table of all unordered pairs (p, q) with $p \neq q$. For n states, there are $\frac{n(n-1)}{2}$ pairs.
3. **Step 2: Base Initialization (Distinguish Final from Non-Final):** For every pair (p, q) , if one state is in F and the other is NOT in F , mark the cell with an **X** (witness string is ϵ).
4. **Step 3: Iterative Propagation:** For every unmarked pair (p, q) :
 - For each symbol $a \in \Sigma$, compute $p' = \delta(p, a)$ and $q' = \delta(q, a)$.
 - If $p' \neq q'$ and the pair (p', q') is already marked with an **X**, mark (p, q) with an **X**.

Repeat Step 3 until an entire pass completes with no new marks added (fixed point reached).

5. **Step 4: Group Equivalent States:** All pairs (p, q) that remain unmarked are equivalent ($p \equiv q$). Partition states into equivalence classes.
6. **Step 5: Construct Minimized DFA:** Each equivalence class forms a single composite state. The start state is the class containing q_0 . Accepting states are classes containing any original final state.

State	Input 0	Input 1
$\rightarrow A$	B	C
B	B	D
C	C	C

State	Input 0	Input 1
* D (Final)	B	E
E	B	C

Before and After DFA State Diagrams

Original (5 States) vs Minimized (3 States) DFA

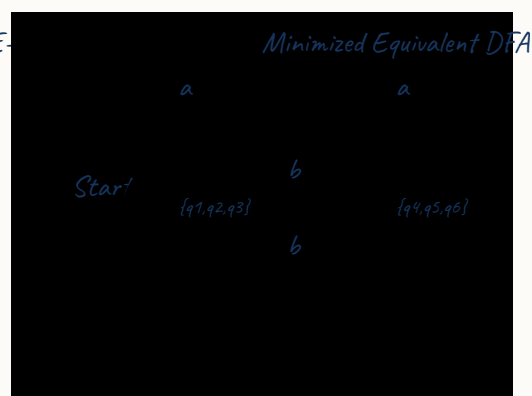
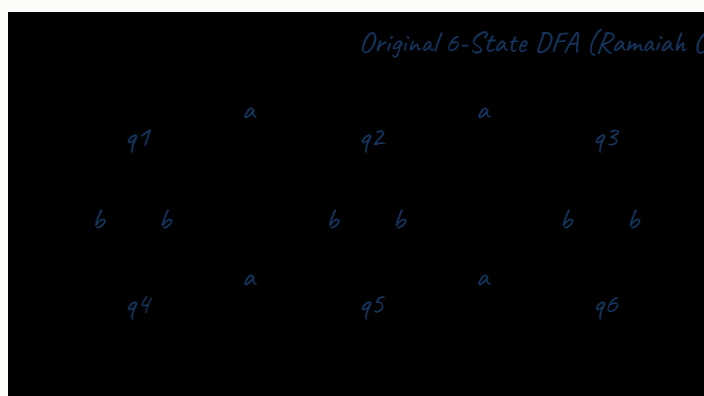
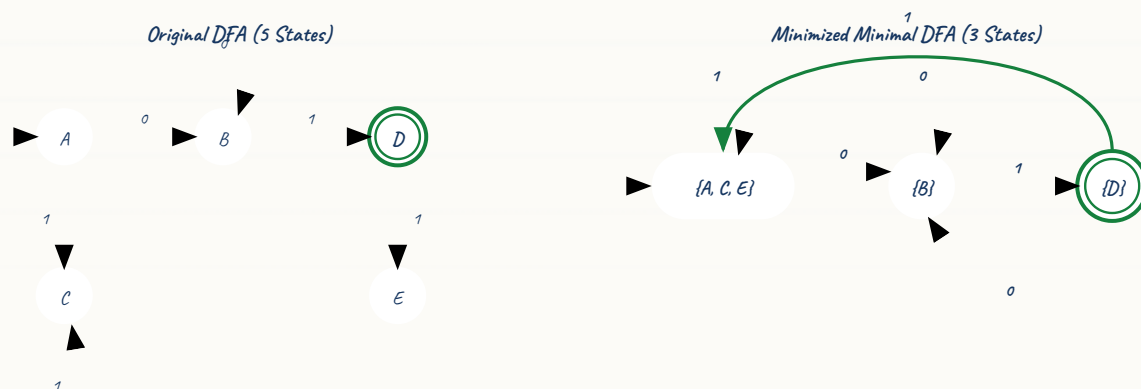


Figure 2.1: 6-State DFA Minimization to 2 States (Ramaiah CIE-1 Q1.b Solved)

Accept states $\{q1, q2, q3\}$ and non-accept states $\{q4, q5, q6\}$ form two distinct equivalence classes under input 'a' and 'b'.

[Interactive Visualizer Online]: Interactive DFA Minimization (Table-Filling Algorithm). Explore the step-by-step visualizer on the live course notes page.

QUICK RECALL

- Step 0: Prune unreachable states.
- Base: Mark all (f, q) where $f \in F, q \notin F$.
- Induction: For (p, q) , if for some symbol a , $(\delta(p, a), \delta(q, a))$ is marked, mark (p, q) .
- Unmarked cells form equivalence classes $[p]$ that become states in the minimal DFA.

One-Page Rapid Revision Sheet

REVISION

Key Identities & Laws

- $R + \emptyset = R, R\epsilon = R, R\emptyset = \emptyset$
- $(R^*)^* = R^*, \emptyset^* = \epsilon, \epsilon^* = \epsilon$
- $(R + S)^* = (R^*S^*)^* = (R^* + S^*)^*$
- $R(S + T) = RS + RT$ (distributive)
- **Arden's Theorem:** $R = Q + RP \implies R = QP^*$ (when $\epsilon \notin P$)

Pumping Lemma Checklist

- Conditions: $|xy| \leq p, |y| \geq 1, \forall i \geq 0, xy^iz \in L$.
- $a^n b^n$: pick $w = a^p b^p, y = a^k, \text{ pump } i = 2 \implies a^{p+k} b^p \notin L$.
- ww^R : pick $w = 0^p 1 0^p, \text{ pump } i = 2$.
- 0^{n^2} : pick $w = 0^{p^2}, \text{ pump } i = 2 \implies p^2 < |w'| < (p+1)^2$.
- Used only for non-regularity proofs!

Closure & Decision Properties

- Closed under: $\cup, \cap, \cdot, *, \bar{L}, -, L^R, h(L), h^{-1}(L)$.
- Complement: invert final states of **complete DFA**.
- Membership: simulate DFA in $O(|w|)$.
- Emptiness: check reachability from q_0 to F (BFS/DFS).
- Finiteness: check cycles on paths from q_0 to F .

DFA Minimization Summary

- Table-filling pairs: $\frac{n(n-1)}{2}$ cells.
- Base: mark (f, q) where $f \in F, q \notin F$.
- Step: mark (p, q) if $(\delta(p, a), \delta(q, a))$ marked for any $a \in \Sigma$.
- Equivalent states remain unmarked.
- Minimal DFA is unique up to state isomorphism.

Important Examination Questions

? QUESTIONS

2-Mark Questions (Short Answer)

Practice Problem:

2M State Arden's Theorem and its condition of validity.

Solution:

Answer: Let P and Q be two regular expressions. If P does not contain ϵ , then the equation $R = Q + RP$ has a unique solution given by $R = QP^*$. The condition $\epsilon \notin P$ is mandatory to guarantee uniqueness of the solution.

Practice Problem:

2M What are the three conditions of the Pumping Lemma for regular languages?

Solution:

Answer: For a string w with $|w| \geq p$ decomposed as $w = xyz$:

1. $|xy| \leq p$ (pump lies within first p symbols)
2. $|y| \geq 1$ (pump is non-empty)
3. $xy^iz \in L$ for all $i \geq 0$ (pumping preserves membership)

Practice Problem:

2M Why cannot an NFA be complemented simply by flipping its final and non-final states?

Solution:

Answer: An NFA can have multiple computational branches for a single string. If one branch terminates in a rejecting state while another branch terminates in an accepting state, the string is accepted. Flipping states would cause both branches to invert, and the string might still be accepted! Thus, one must determinize into a complete DFA before swapping states.

5-Mark Questions (Medium Length)

Practice Problem:

5M Explain the State Elimination Method to convert an FA to a Regular Expression.

Solution:

Answer:

1. Create a new unique start state s_{new} with an ϵ -transition to the old start state, and a new unique final state f_{new} receiving ϵ -transitions from all old final states.
2. Convert parallel edges between any two states into single edges labeled with the union ($R_1 + R_2$).
3. Eliminate each intermediate state k one by one. For every predecessor p and successor q of k , update edge (p, q) to $\alpha_{pq}^{new} = \alpha_{pq} + \alpha_{pk}(\alpha_{kk})^* \alpha_{kq}$, where α_{kk} is the self-loop on k .
4. When only s_{new} and f_{new} remain, the label on (s_{new}, f_{new}) is the equivalent regular expression.

Practice Problem:

5M Describe the algorithm to decide whether the language of a DFA is finite or infinite.

Solution:

Answer:

1. Eliminate all states unreachable from q_0 using BFS/DFS.
2. Eliminate all "dead" states (states from which no final state in F can ever be reached).
3. In the resulting pruned transition graph, test for the presence of a directed cycle using cycle-detection DFS.
4. If any directed cycle exists, the language is **infinite** (the loop can be traversed arbitrarily many times to generate valid strings). If no cycles exist (it is a DAG), the language is **finite**.

10-Mark Questions (Long Answer / Comprehensive)

Practice Problem:

10M State and prove the Pumping Lemma for regular languages, and use it to prove that $L = \{0^i 1^j \mid i > j\}$ is not regular.

Solution:

Answer Summary:

1. **Statement & Proof:** State the lemma conditions ($|xy| \leq p, |y| \geq 1, \forall i \geq 0, xy^i z \in L$). Proof relies on Pigeonhole Principle on the first $p + 1$ states of a p -state DFA, showing a repeating state $r_j = r_k$ creates a loop of length $|y| = k - j \geq 1$.
2. **Non-regularity Proof for $L = \{0^i 1^j \mid i > j\}$:**
 - Assume L is regular with pumping length p .
 - Choose $w = 0^{p+1} 1^p \in L$ (since $p + 1 > p$). $|w| = 2p + 1 \geq p$.
 - Partition $w = xyz$ with $|xy| \leq p$ and $|y| \geq 1$. Because $|xy| \leq p$, y consists strictly of 0's, so $y = 0^k$ with $1 \leq k \leq p$.
 - Pump down using $i = 0$: $w' = xy^0 z = xz$.
 - Length of 0's in w' becomes $(p + 1) - k$. Since $k \geq 1$, $(p + 1) - k \leq p$.
 - Now the number of 0's is $\leq p$, while the number of 1's remains p . The condition $i > j$ no longer holds! Hence $w' \notin L$.
 - Contradiction! Therefore L is not regular.

Solved Questions from Past CIE-1 & SEE Examination Papers

ARCHIVE

The following examination questions have been transcribed from the official Ramaiah Institute of Technology papers in notes/toc/practice/toc-cie-1-2-see.pdf (CIE-1 Oct 2025 and SEE Jan 2026), solved with complete step-by-step mathematical rigor:

Ramaiah Internal Assessment – I (Oct 2025) · Q 1(b) [8 Marks · Blooms L3, CO2]

"Minimize DFA for the following state diagram: States $\{q_1, q_2, q_3, q_4, q_5, q_6\}$, Start: q_1 , Accept: $\{q_1, q_2, q_3\}$. Transitions: q_1 on $a \rightarrow q_2$, on $b \rightarrow q_4$; q_2 on $a \rightarrow q_3$, on $b \rightarrow q_5$; q_3 on $a \rightarrow q_2$, on $b \rightarrow q_6$; q_4 on $a \rightarrow q_5$, on $b \rightarrow q_1$; q_5 on $a \rightarrow q_6$, on $b \rightarrow q_2$; q_6 on $a \rightarrow q_5$, on $b \rightarrow q_3$."

Step 1: Partition into 0-Equivalence (Accept vs Non-Accept):

$$P_0 = \{\{q_1, q_2, q_3\}, \{q_4, q_5, q_6\}\}$$

Step 2: Compute 1-Equivalence on Group $\{q_1, q_2, q_3\}$:

- $\delta(q_1, a) = q_2 \in \{q_1, q_2, q_3\}, \delta(q_1, b) = q_4 \in \{q_4, q_5, q_6\}$
- $\delta(q_2, a) = q_3 \in \{q_1, q_2, q_3\}, \delta(q_2, b) = q_5 \in \{q_4, q_5, q_6\}$
- $\delta(q_3, a) = q_2 \in \{q_1, q_2, q_3\}, \delta(q_3, b) = q_6 \in \{q_4, q_5, q_6\}$

All three states transition to the exact same group on 'a' (Group 1) and 'b' (Group 2). Hence $\{q_1, q_2, q_3\}$ cannot be partitioned!

Step 3: Compute 1-Equivalence on Group $\{q_4, q_5, q_6\}$:

- $\delta(q_4, a) = q_5 \in \{q_4, q_5, q_6\}, \delta(q_4, b) = q_1 \in \{q_1, q_2, q_3\}$
- $\delta(q_5, a) = q_6 \in \{q_4, q_5, q_6\}, \delta(q_5, b) = q_2 \in \{q_1, q_2, q_3\}$
- $\delta(q_6, a) = q_5 \in \{q_4, q_5, q_6\}, \delta(q_6, b) = q_3 \in \{q_1, q_2, q_3\}$

All three non-accept states likewise transition identically. Thus $P_1 = P_0$. Fixed point reached!

Final Minimized DFA (2 States):

- State $A = \{q_1, q_2, q_3\}$ (Start & Accept) · $\delta(A, a) = A, \quad \delta(A, b) = B$
- State $B = \{q_4, q_5, q_6\}$ (Non-Accept) · $\delta(B, a) = B, \quad \delta(B, b) = A$

(Complete before-and-after state diagrams illustrated in Figure 2.1 above).

Ramaiah Internal Assessment – I (Oct 2025) · Q 2(b) [7 Marks · Blooms L3, CO2]

"Define regular expressions. Convert the following regular expressions to NFA with ϵ -Transitions:

i) $(00)^* 11 (0+1)^*$, ii) $a^* + b^* + c^*$."

Definition: A Regular Expression (RE) over alphabet Σ is a formal algebraic string defining a regular language built inductively using atomic base cases ($\emptyset$, ϵ , $a \in \Sigma$) and three closed operators: Union ($R_1 + R_2$), Concatenation ($R_1 R_2$), and Kleene Star (R^*).

Part (i): Thompson Construction for $(00)^* 11 (0 + 1)^*$:

1. Sub-NFA for 00 : Initial state $s_0 \xrightarrow{0} s_1 \xrightarrow{0} s_2$.
2. Kleene Star $(00)^*$: Add start state s_{in} with ϵ -transition bypassing to final state, and feedback ϵ -transition from $s_2 \rightarrow s_0$.
3. Concatenate with 11 : Chain via $\epsilon \rightarrow s_3 \xrightarrow{1} s_4 \xrightarrow{1} s_5$.
4. Concatenate with $(0 + 1)^*$: Branch parallel paths for 0 and 1 wrapped in Kleene star loop (illustrated in Figure 2.2).

Part (ii): Thompson Construction for $a^* + b^* + c^*$:

Create a single master start state q_0 that branches via ϵ -transitions to three parallel sub-automata: M_a for a^* , M_b for b^* , and M_c for c^* . The accept states of all three sub-automata merge via ϵ -transitions into a single final accept state q_f .

State	Input 0	Input 1	Role
$\rightarrow A$	B	A	Start State
B	A	C	Intermediate
C	D	B	Pre-accept
* D	D	A	Accept State

Practice Problems with Step-by-Step Solutions

PRACTICE

Test your mastery of regular expressions, pumping lemmas, and DFA minimization with these exam-standard numericals. Click each card to reveal the verified solution:

PRACTICE 1 · REGULAR ALGEBRA & ARDEN

Solve the equation $R = aR + b$ using Arden's theorem and prove that the solution is unique when $\epsilon \notin P$.

Practice Problem:

Show Step-by-Step Solution

Solution:

Arden's Theorem states that if P and Q are regular expressions and $\epsilon \notin P$, then the equation $R = Q + RP$ (or $R = PR + Q$) has a unique solution $R = P^*Q$ (or $R = QP^*$).

For $R = aR + b$, identify $P = a$ and $Q = b$. Since $\epsilon \notin a$:

$$R = a^*b$$

Uniqueness Proof: Repeated substitution gives $R = a(aR + b) + b = a^2R + ab + b = \dots = a^kR + a^{k-1}b + \dots + b$. In the limit as $k \rightarrow \infty$, a^*b is the only finite string sequence satisfying the equality.

PRACTICE 2 · PUMPING LEMMA FOR PALINDROMES

Prove that the language of even-length palindromes $L = \{ww^R \mid w \in \{0,1\}^*\}$ is not regular using the Pumping Lemma.

Practice Problem:

Show Step-by-Step Solution

Solution:

1. Assume L is regular with pumping length p .
2. Select test string $s = 0^p110^p \in L$. Length $|s| = 2p + 2 \geq p$.
3. By condition $|xy| \leq p$, y must be entirely within the first p zeros ($y = 0^k$ for $k \geq 1$).
4. Pump with $i = 2$: $s' = xy^2z = 0^{p+k}110^p$.
5. For s' to belong to L , the number of leading zeros must equal trailing zeros ($p + k = p$), implying $k = 0$, which contradicts $|y| \geq 1$.
6. Hence $s' \notin L$, proving L is not regular.

PRACTICE 3 · NON-LINEAR LENGTH PUMPING

Show that the language $L = \{0^{n^2} \mid n \geq 1\}$ (strings whose lengths are perfect squares) is not regular.

Practice Problem:

Show Step-by-Step Solution

Solution:

1. Assume L is regular with pumping length p .
2. Pick $s = 0^{p^2} \in L$. Decompose $s = xyz$ where $|xy| \leq p$ and $1 \leq |y| \leq p$.
3. Pump with $i = 2$: $|s'| = |xy^2z| = |s| + |y| = p^2 + |y|$.
4. Since $1 \leq |y| \leq p$, we have $p^2 < p^2 + |y| \leq p^2 + p < (p+1)^2 = p^2 + 2p + 1$.
5. $|s'|$ falls strictly between two consecutive perfect squares p^2 and $(p+1)^2$.
Hence its length cannot be a square!
6. Therefore $s' \notin L$, establishing that L is not regular.

PRACTICE 4 · CLOSURE UNDER REVERSAL

Prove that if language L is regular, then its reversal $L^R = \{w^R \mid w \in L\}$ is also regular.

Practice Problem:

Show Step-by-Step Solution

Solution:**Proof via Finite Automata:**

1. Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA accepting L .
2. Construct NFA $M^R = (Q \cup \{q_{new}\}, \Sigma, \delta^R, q_{new}, \{q_0\})$.
3. Reverse all transition arrows: if $\delta(p, a) = q$, then $p \in \delta^R(q, a)$.
4. Create new start state q_{new} with ϵ -transitions to all states in F .
5. Set original start state q_0 as the sole accepting state of M^R .
6. A string w is accepted by M^R if and only if $w^R \in L(M)$. Thus L^R is regular.

PRACTICE 5 · DECISION PROPERTY: EMPTINESS

Describe an algorithm to decide whether the language accepted by a DFA M is empty ($L(M) = \emptyset$). What is its computational complexity?

Practice Problem:

Show Step-by-Step Solution

Solution:

Algorithm: Graph Reachability (Breadth-First Search / Depth-First Search).

1. Treat DFA state transition diagram as a directed graph $G = (V, E)$ where vertices are states Q and edges are transitions.
2. Execute BFS or DFS starting from start state q_0 to find all reachable states Q_{reach} .
3. Test intersection: if $Q_{reach} \cap F = \emptyset$, then $L(M) = \emptyset$ (Language is EMPTY). Otherwise, language is non-empty.
4. **Complexity:** $O(|V| + |E|) = O(|Q| + |Q| \cdot |\Sigma|) = O(|Q| \cdot |\Sigma|)$, which is linear in the size of the transition table.

PRACTICE 6 · EQUIVALENCE TESTING

Explain how the table-filling algorithm can be used to decide whether two DFAs M_1 and M_2 accept the exact same language ($L(M_1) = L(M_2)$).

Practice Problem:

Show Step-by-Step Solution

Solution:

1. Assume $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ over the same alphabet Σ , with disjoint state sets ($Q_1 \cap Q_2 = \emptyset$).
2. Form union automaton $M_{12} = (Q_1 \cup Q_2, \Sigma, \delta_1 \cup \delta_2, \dots, F_1 \cup F_2)$.
3. Run the Table-Filling algorithm on M_{12} across all pairs in $(Q_1 \cup Q_2)$.
4. Inspect pair (q_{01}, q_{02}) : if (q_{01}, q_{02}) remains **unmarked**, then the start states are equivalent, meaning $L(M_1) = L(M_2)$.
5. If (q_{01}, q_{02}) is marked, $L(M_1) \neq L(M_2)$.